



DATABASES – HANDS-ON SQL

SQL PRACTICAL COURSE

Under construction – more Units follow !

PROF. DR. RER. NAT.
ALEXANDER VOß
FH AACHEN



UNIT 0x00 GENERAL INFORMATION



A FEW NOTES IN ADVANCE

In this course, the text refers to attributes, columns, tables, keys, schemas or databases, just to name a few. Here is a brief definition of the terms.

- Attributes, tables: The table on the right named shop_product models products from the AMI_ZONE company. The columns of the table are the so-called attributes, i.e. the properties of each product. Each row is an entity/object - here a product.
- Standard data types in PostgreSQL: Attributes are typed, e.g.
 - integer (int): classic integer
 - numeric(p,s): exact decimal number with p digits in total and s after the point, e.g. prices
 - double precision: floating point
 - varchar(n): text with at most n characters;
 - text: text of any length
 - timestamp: date and time in the form YYYY-MM-DD hh:mm:ss; date and time hold only one part
 - boolean: true or false

	name	cat...	unit
1	Spinach		1 PK
2	Four Cheese Pizza		1 PC
3	Spinach Pizza		1 PC
4	Fish Fingers		1 PK
5	Pasta Pan		1 PK
6	Carrots		2 KG
7	Onions		2 KG
8	Bananas		2 KG

Products in AMI_ZONE



A FEW NOTES IN ADVANCE

- Key, id: An entity almost always has some kind of unique identifier or id, also called a key. This identifies the specific entity and is unique.
- Foreign key: An attribute in one table that represents a key in another table is called a foreign key. The exact definition and meaning follows.
- Schema/Database: Tables are grouped together in a schema. In PostgreSQL a server holds several databases, and each database several schemas. We work in the database postgres with the schemas ami_zone, ami_kemper, ... In other DBMS "database" and "schema" may mean the same.
- Indexes: Additional data structures, e.g. B-trees, are created automatically or intentionally to allow quick access to data.

id	name
1	Frozen Goods
2	Fruits, Vegetables
3	Sausages, Cold Meat
4	Milk Products
5	Cold Drinks
6	Barbecue Products
7	Snacks
8	Hot Drinks
9	Convenience Foods
10	Baked Goods

id	name
4	Fish Fingers
5	Pasta Pan
6	Carrots
7	Onions
8	Bananas
9	Garlic
10	Pork Sausage
11	Meatballs
12	Milk
13	Quark

Products and Categories



A FEW NOTES IN ADVANCE

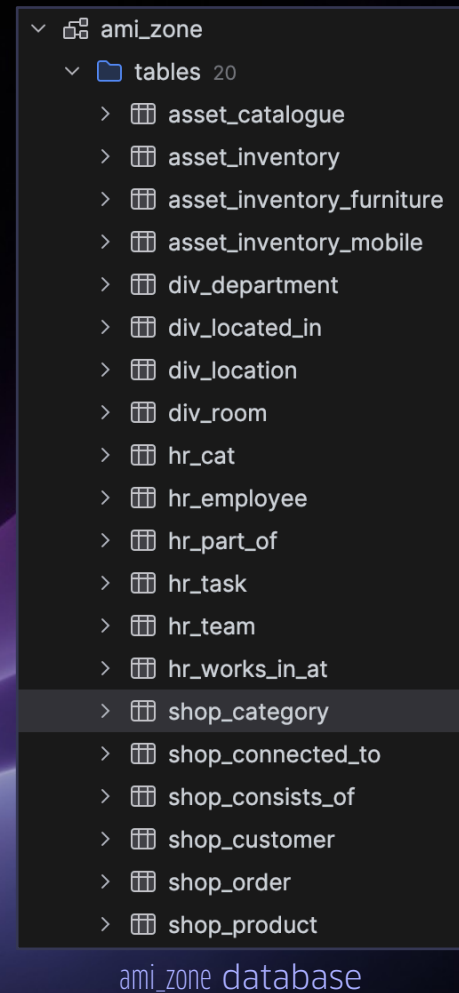
- Upper/lower case: In PostgreSQL, keywords are case-insensitive (SELECT = select). Unquoted names are folded to lower case (Product → product); names in double quotes keep their case. String comparisons are case-sensitive: 'Pizza' ≠ 'pizza'. It is not relevant to the exams.
- NULL: In addition to the typical range of values for each data type, there is another value that an attribute can have: NULL. The meaning of the value NULL is given by the context, often the value is simply used for "no specification" or "not yet specified".
The column definition of a table may allow or disallow NULL, but if the value is allowed, it often requires special handling in queries.
- Script: The SQL commands for a unit are also available as an sql file, so you can try them out instead of typing them in.



SAMPLE DATA AMI_ZONE

The Idea

- AMI_ZONE is a fictitious company with individual departments and employees at different locations that delivers everyday necessities when there is no time to go shopping.
Typical examples are frozen pizzas or the latest c't magazine.
- The database contains a lot of data and relationships and is used to practise SQL commands.





UNIT 0x01

SELECTION & PROJECTION



SELECT & SEARCH_PATH

EXAMPLES

Intention

Query data.

Projection and Selection

- *Projection* selects columns by specifying table attributes.
- *Selection* filters rows by specifying a criterion that must be met for each row.

SEARCH_PATH (default schema)

- If a default schema or a way to find a table is specified with `SEARCH_PATH` the explicit specification of the schema can be omitted for tables in SQL expressions.

```
-- all data
SELECT * FROM ami_zone.shop_product;

-- default schema, i.e. unqualified table names refer to ami_zone
SET SEARCH_PATH = ami_zone;

-- projection example
SELECT name, price FROM shop_product;

-- selection example
SELECT * FROM shop_product WHERE price>2;
```

	name	price
1	Spinach	1.99
2	Four Cheese Pizza	2.39
3	Spinach Pizza	2.29
4	Fish Fingers	1.99
5	Pasta Pan	3.29
6	Carrots	0.39

Projection



SELECTION * & TABLE ALIAS

EXAMPLES

Intention

Query all records and attributes of a table, and use a table alias.

Select All

- * selects all attributes in a table.
- The column headers are the attribute names or columns of the table.
- The order of the records is not specified here.

Table Alias

- Tables can be given an alias name, here P, which is used instead.
- This not only makes more complex expressions shorter, but also removes any ambiguity.
- Attributes in the query then may get or need this prefix.

```
-- tables with alias
SELECT P.* FROM shop_product P;

-- attributes with alias
SELECT P.name, P.price FROM shop_product P WHERE price>2;
```

name	price
Four Cheese Pizza	2.39
Spinach Pizza	2.29
Pasta Pan	3.29
Meatballs	2.35
Grill Sausages	3.55

Selection and Projection



PROJECTION & COLUMN ALIAS

EXAMPLE

Intention

Query only certain attributes (projection).

Column names

- Column names, e.g. ID, can be specified explicitly.
- AS is optional.

Quotes in PostgreSQL

- single quotes are string literals ('Pizza');
- double quotes are names, which keep their case and may contain blanks ("Price [€]").
- Unquoted names are folded to lower case: name Product gives the column product.

```
-- note column names (aliases): unquoted names are folded to lower case
-- (Product -> product),
-- double quotes keep case and allow blanks; single quotes are string literals,
-- not names
SELECT id as "IDX",
       name Product,
       price "Price [€]"
FROM shop_product;
```

IDX	product	Price [€]
1	Spinach	1.99
2	Four Cheese Pizza	2.39
3	Spinach Pizza	2.29
4	Fish Fingers	1.99
5	Pasta Pan	3.29
6	Carrots	0.39

Column Alias



FUNCTIONS

EXAMPLE

Intention

Use arithmetic expressions and functions (e.g. round, concat).

Documentation

- Consulting the documentation is the best way to find out about the calling specification and possible variations of similar functions.

```
-- calculated values
SELECT
  id "ID ",
  concat(name, ' [' , unit, ']') "Product",
  price "Price [€]",
  round(price/(1.0+vat),2) "excl. VAT",
  round(price/(1.0+vat)*vat,2) "VAT "
FROM shop_product;
```

"ID "	"Product"	"Price [€]"	"excl. VAT"	"VAT "
1	Spinach [PK]	1.99	1.86	0.13
2	Four Cheese Pizza [PC]	2.39	2.23	0.16
3	Spinach Pizza [PC]	2.29	2.14	0.15

Functions



OPERATORS & NULL

EXAMPLES

Intention

Use expressions such as `is NULL` and functions with `NULL`.

NULL

- Caution: Expressions are `NULL` if one of the operands is `NULL`.
- `is NULL` returns a boolean (`true/false`).
- `coalesce` returns the first non-`NULL` parameter.

```
-- calculated values involving NULL
SELECT
    brand,
    risk_of_loss_percent,
    risk_of_loss_percent is NULL "NULL?",
    coalesce(100-risk_of_loss_percent,100.00) "Fulfilment"
FROM shop_customer;
```

brand	risk_of_loss_percent	"NULL?"	"Fulfilment"
Deutsche Bank	<null>	• true	100
Börse	<null>	• true	100
E.ON	<null>	• true	100
Infineon	<null>	• true	100
Lufthansa	<null>	• true	100
RWE	2.00	false	98

Expressions with NULL



DISTINCT

EXAMPLES

Intention

Duplicate data should not be included.

Remarks

- Without distinct results may occur more than once. This is where the analogy with a mathematical set breaks down.
- Further attributes, e.g. name, usually do not make sense in the second query - why?

```
-- distinct values  
SELECT category_id, name FROM shop_product;  
SELECT distinct category_id FROM shop_product;
```

category_id	name
1	Spinach
1	Four Cheese Pizza
1	Spinach Pizza
1	Fish Fingers
1	Pasta Pan
2	Carrots
2	Onions

category_id
11
9
3
5
4
10
6

With and without distinct



COUNT

EXAMPLES

Intention

Determine the number of records.

Remarks

- A * in count causes all records to be counted, whereas specifying an attribute causes only records other than NULL to be counted.
- Adding distinct considers only different values.

```
-- count number of lines, with or without NULLs
select count(*) from shop_customer;
select count(discount_percent) from shop_customer;
select count(distinct discount_percent) from shop_customer;
select count(risk_of_loss_percent) from shop_customer;
```

count	count	count	count
12	12	4	1

count



WHERE

EXAMPLE

Intention

Query only table attributes or expressions matching a condition (after where).

Remarks

- The condition, here `price=1.99`, tests for equality. `price` is of type `numeric(12,2)`, an exact decimal type, so this is safe. With floating-point types (real, double precision) the usual caveats about rounding errors apply.

```
-- selection with WHERE
SELECT name, price FROM shop_product
WHERE price=1.99;
```

	name	price
1	Spinach	1.99
2	Fish Fingers	1.99
3	Pork Sausage	1.99
4	Chips	1.99
5	Pasta in Sauce	1.99

where



OPERATORS

EXAMPLES

Intention

Use operators such as like, not, and in where conditions.

Like

like is used for string comparison; like is case-sensitive in PostgreSQL; ilike ignores case.

Wildcards

The % wildcard allows any number of characters (including none),

The _ wildcard allows exactly one character.

```
-- string comparison with like including wildcards % and _
SELECT name, price FROM shop_product
  WHERE name like '%Pizza';
SELECT name, price FROM shop_product
  WHERE name like '%Pizza' and
  not name like '%Spinach%';
```

like

name	price
Four Cheese Pizza	2.39
Spinach Pizza	2.29

name	price
Four Cheese Pizza	2.39



OPERATORS

EXAMPLES

Intention

Use operators such as between...and, in in where conditions.

Range

- The boundaries at between...and are inclusive.

```
-- between and in
SELECT name, price FROM shop_product
    WHERE price between 1.00 and 2.00;
SELECT name, price FROM shop_product
    WHERE price in (0.99, 1.99, 2.99);
```

name	price	name	price
Spinach	1.99	Spinach	1.99
Fish Fingers	1.99	Fish Fingers	1.99
Bananas	1.29	Pork Sausage	1.99
Pork Sausage	1.99	Quark	0.99
Cola light	1.50	Water	0.99

between...and & in



CASE

EXAMPLE

Intention

The case statement allows an output per case.

```
-- case
SELECT name, price,
       case
         when price <= 2.00 then 'cheap'
         when price <= 5.00 then 'ok'
         else 'expensive'
       end as affordable
FROM shop_product
WHERE price between 1.00 and 10.00;
```

name	price	affordable
Spinach	1.99	cheap
Four Cheese Pizza	2.39	ok
Spinach Pizza	2.29	ok
Fish Fingers	1.99	cheap
Pasta Pan	3.29	ok

case



AND & OR

EXAMPLES

Intention

Use operators such as and, or, is NULL, is not NULL and parentheses in where conditions.

NULL

- As before, be careful with NULL values.
The result of an arithmetic operation with NULL is also NULL.

```
-- different clauses
SELECT name, price FROM shop_product
  WHERE (price<1.00 or price>5.00)
    and category_id=11;
SELECT brand, risk_of_loss_percent FROM shop_customer
  WHERE risk_of_loss_percent is not null;
```

and & or & is not NULL

brand	risk_of_loss_percent
RWE	2.00

name	price
Bild	0.90
GameStar	6.50



SORT BY

EXAMPLES

Intention

Sort the found results by criteria (order by), ascending (asc), or descending (desc).

Asc & Desc

- In the lower result, products with the same price are additionally sorted by name, descending.

```
-- order by, asc, desc
SELECT name, price FROM shop_product
    WHERE category_id in (1,3,4) ORDER BY price; -- asc
SELECT name, price FROM shop_product
    WHERE category_id in (1,3,4) ORDER BY price, name desc;
```

name	price	name	price
Milk	0.79	Milk	0.79
Yoghurt	0.98	Yoghurt	0.98
Quark	0.99	Quark	0.99
Pork Sausage	1.99	Spinach	1.99
Fish Fingers	1.99	Pork Sausage	1.99
Spinach	1.99	Fish Fingers	1.99
Spinach Pizza	2.29	Spinach Pizza	2.29
Meatballs	2.35	Meatballs	2.35
Four Cheese Pizza	2.39	Four Cheese Pizza	2.39
Pasta Pan	3.29	Pasta Pan	3.29

order by



INTERACTIVE

EXAMPLE

Intention

Interactive use of select.

Remarks

- In PostgreSQL a select needs no FROM clause.
Some DBMS, such as Oracle, require a table and provide the pseudo table dual for this; PostgreSQL has no dual.

```
-- interactive usage, a SELECT does not need a FROM clause in PostgreSQL  
-- (Oracle needs 'FROM dual' here, PostgreSQL does not know DUAL)  
select now(), 1+2*3, random();
```

☐ now	☐ ?column?	☐ random
2026-10-06 20:40:12.393198 +00:00	7	0.5180233623461439

Some expression



CHECK

Tasks

1. Print the entire contents of each of the tables `div_department`, `shop_customer`, `hr_employee`, `shop_product`, `hr_team` and `shop_category` from `ami_zone`.
2. Find out how many different VAT rates there are in the `shop_product` table and print them.
3. Which customers are a *Bank* or *Sparkasse* by name? Identify them and print only the name and discount. Remember: `like` is case-sensitive, `ilike` is not.
4. Find the monthly salary (assume 12 months) of all employees whose manager is Mia or Ben. Give the column the heading `Monthly salary` and round the monthly salary to full Euros. (Look up the id of Mia and Ben, no sub-query required yet).
5. How many employee names begin with an M? And how many contain an M? Find out the number of each.



UNIT 0x02

JOINS